

Bases de Datos

Clase 15: Algoritmos de Join (I)

Join: la operación más costosa

```
SELECT C.cnombre, B.bnombre  
FROM Capitanes C, Botes B, Reservas R  
WHERE C.cid = R.cid AND R.bid = B.bid
```

- El join combina tuplas de **dos o más** tablas
- Es la operación más cara de SQL
- Supondremos joins de **igualdad** (equi-joins): $R.a = S.a$
- Existen **múltiples algoritmos** con costos muy diferentes

Notación para esta clase

Símbolo	Significado
Páginas(R), Páginas(S)	Número de páginas de cada relación
Tuplas(R), Tuplas(S)	Número de tuplas de cada relación
B	Páginas disponibles en buffer

Siempre medimos costo en **número de I/O** (páginas leídas/escritas).

Nested Loop Join

Nested Loop Join (NLJ): idea

```
for each r ∈ R:           // outer loop
  for each s ∈ S:         // inner loop
    if r.a = s.a then
      emitir (r, s)
```

Para cada tupla de R, recorrer **todas** las tuplas de S y verificar la condición.

Es la implementación más directa: un **doble loop**.

NLJ: costo

- Leemos R una vez: **Páginas(R)**
- Por cada **tupla** de R, leemos S entera: **Tuplas(R) × Páginas(S)**

$$\text{Costo NLJ} = \text{Páginas}(R) + \text{Tuplas}(R) \times \text{Páginas}(S)$$

NLJ: ejemplo numérico

R y S son tablas de **16 MB** cada una, páginas de **8 KB**, tuplas de **300 bytes**.

$$\begin{aligned}\text{Páginas}(R) &= \text{Páginas}(S) = 16 \text{ MB} / 8 \text{ KB} = 2.048 \\ \text{Tuplas}(R) &\approx 16 \text{ MB} / 300 \text{ B} \approx 55.000\end{aligned}$$

$$\text{Costo} = 2.048 + 55.000 \times 2.048 \approx 112.642.048 \text{ I/O}$$

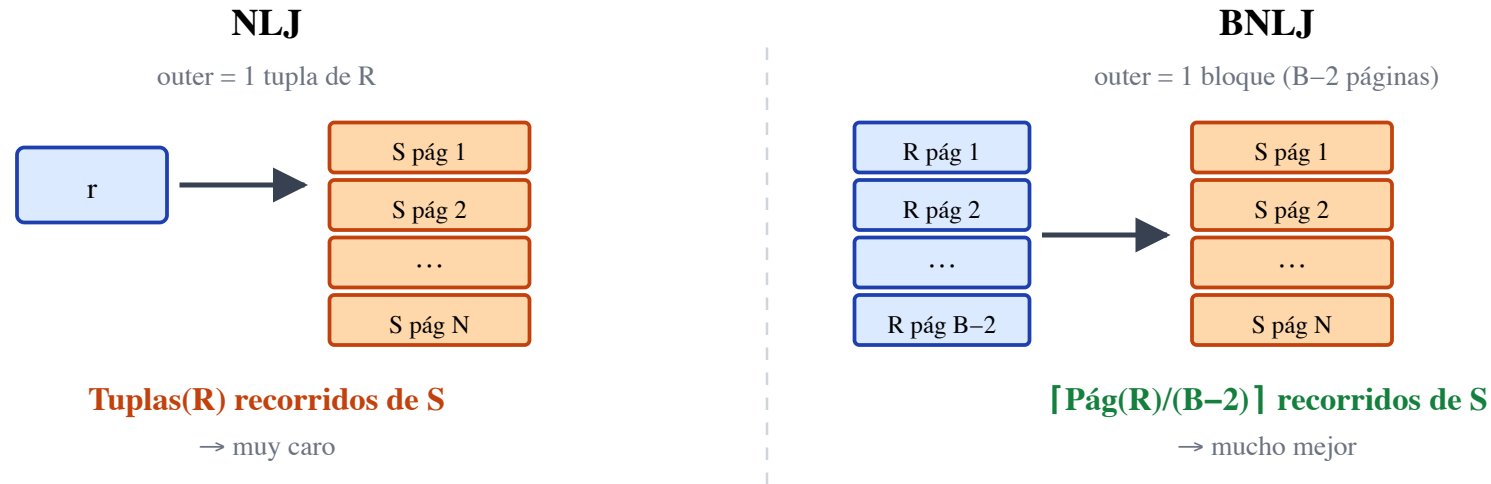
A **0,1 ms** por I/O $\rightarrow \approx 3,1$ horas

Sobre costos

- El costo depende de **cuál tabla es la outer** (R) y cuál es la inner (S)
- Conviene poner la tabla **más pequeña** como outer (menos tuplas = menos recorridos de S)
- Pero aun así, el costo es enorme. ¿Podemos hacerlo mejor?

Block Nested Loop Join

Block Nested Loop Join (BNLJ): idea



En vez de recorrer S una vez por cada **tupla** de R, es una vez por cada **bloque** de R.

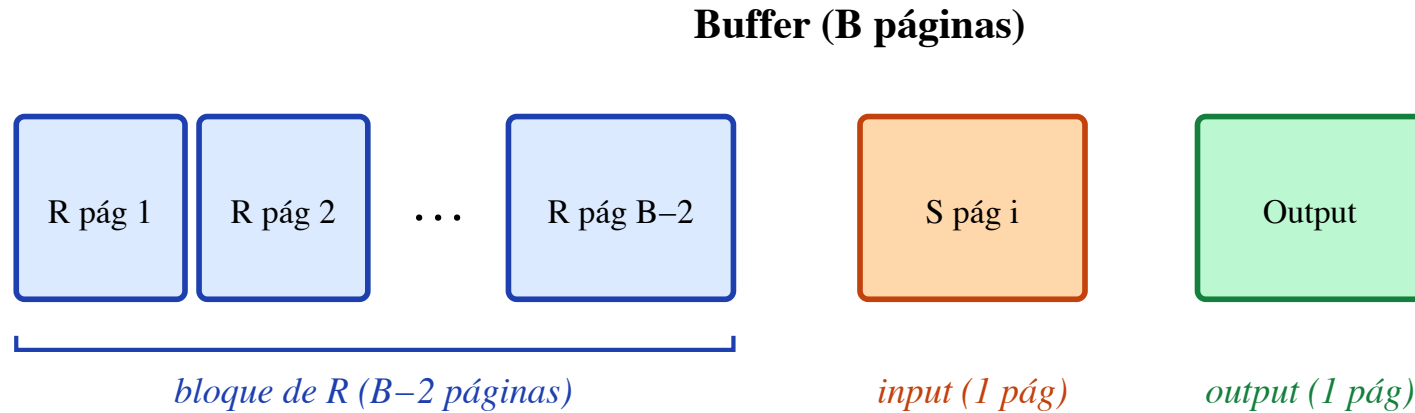
- Llenamos el buffer con páginas de R
- Para cada página de S, la comparamos contra **todas** las tuplas del bloque de R en buffer
- Cuando el bloque se agota → cargar el siguiente bloque de R, reiniciar S

BNLJ: pseudocódigo

```
while quedan páginas de R por leer:
    cargar B-2 páginas de R al buffer           // "bloque" de R
    for each página P_S de S:
        for each tupla s ∈ P_S:
            for each tupla r en el bloque de R:
                if r.a = s.a then
                    emitir (r, s)
```

El loop externo avanza **bloque por bloque** sobre R. Dentro de cada bloque, leemos S una sola vez — cada página de S entra al buffer y se compara contra **todo el bloque** de R antes de ser reemplazada.

BNLJ: layout del buffer



1. Cargar B-2 páginas de R al buffer
2. Leer S página por página; para cada página de S, comparar contra todo el bloque de R
3. Cuando S termina → cargar el siguiente bloque de R, reiniciar S

BNLJ: costo

$$\text{Costo BNLJ} = \text{Páginas}(R) + \left\lceil \frac{\text{Páginas}(R)}{B - 2} \right\rceil \times \text{Páginas}(S)$$

- Leemos R una vez
- Por cada **bloque** de R (hay $\lceil \text{Páginas}(R)/(B-2) \rceil$ bloques), recorreremos S entera

La mejora: pasamos de Tuplas(R) recorridos de S a $\lceil \text{Páginas}(R)/(B-2) \rceil$ recorridos.

BNLJ: ejemplo numérico

Mismas tablas: R, S de 16 MB, páginas de 8 KB. Buffer de **1 MB** (128 páginas).

Páginas(R) = Páginas(S) = 2.048
B = 128 → B-2 = 126 páginas para R
Bloques de R = $\lceil 2.048 / 126 \rceil = 17$

$$\text{Costo} = 2.048 + 17 \times 2.048 = 2.048 + 34.816 = 36.864 \text{ I/O}$$

A 0,1 ms por I/O → **≈ 3,7 segundos** (vs. 3,1 horas con NLJ!)

NLJ vs. BNLJ: comparación visual

[Tabla comparativa lado a lado]

	NLJ	BNLJ
Granularidad del loop externo	1 tupla	1 bloque (B–2 páginas)
Recorridos de S	Tuplas(R)	$\lceil \text{Páginas(R)} / (\text{B} - 2) \rceil$
Ejemplo (16 MB, 1 MB buffer)	3,1 horas	3,7 segundos
¿Usa bien el buffer?	No	Sí

BNLJ: observaciones

1. **Más buffer** → **menos recorridos de S** → más rápido
2. Si **toda R cabe en buffer** ($\text{Páginas}(R) \leq B-2$) → solo **1 recorrido de S**
 - $\text{Costo} = \text{Páginas}(R) + \text{Páginas}(S)$ — el mínimo posible
3. Sigue conviniendo poner la tabla **más pequeña** como outer (R)
4. ¿Y si S tiene un **índice** en el atributo de join? → Index Nested Loop Join

Index Nested Loop Join

Index Nested Loop Join (INLJ): idea

```
Para cada tupla r en R:  
  Usar el índice de S para buscar todas las tuplas s  
    donde  $s.a = r.a$   
  Para cada s encontrada:  
    emitir (r, s)
```

- Supongamos que S tiene un **índice** en el atributo de join
- En vez de recorrer toda S para cada tupla de R, **buscamos en el índice**
- El índice nos lleva directamente a las tuplas de S que hacen match

INLJ: costo

$$\text{Costo INLJ} = \text{Páginas}(R) + \text{Tuplas}(R) \times \text{CostoBúsqueda}(S)$$

Donde **CostoBúsqueda(S)** depende del tipo de índice:

- **B+ Tree en S:** $\approx 2-4$ I/O por búsqueda (profundidad del árbol)
- **Hash Index en S:** $\approx 1-2$ I/O por búsqueda

INLJ: ejemplo numérico (caso analítico)

R de 16 MB (2.048 páginas, ~55.000 tuplas), S con B+ Tree de profundidad 3, buffer de 128 páginas.

$$\text{Costo INLJ} = 2.048 + 55.000 \times 3 = 167.048 \text{ I/O} \approx 16,7 \text{ seg}$$

Algoritmo	Costo I/O	Tiempo
NLJ	112.642.048	3,1 horas
BNLJ	36.864	3,7 seg
INLJ	167.048	16,7 seg

Con tablas grandes y **sin filtros**, BNLJ gana. 55.000 lookups al índice son demasiados.

INLJ: ejemplo numérico 2 -- caso selectivo.

¿Y si R es **pequeña** porque ya aplicamos un filtro?

Ejemplo: tras `WHERE ciudad = 'Santiago'`, R se reduce a 5 páginas (100 tuplas).

- **BNLJ**: R cabe en el buffer $\rightarrow$ 1 scan completo de S $\rightarrow 5 + 2.048 = 2.053$ I/O ≈ 205 ms
- **INLJ**: $5 + 100 \times 3 = 305$ I/O ≈ 30 ms

Caso extremo — consulta puntual (`WHERE c.id = 42` $\rightarrow$ R = 1 tupla):

$$\text{Costo INLJ} = 1 + 1 \times 3 = 4 \text{ I/O} \approx 0,4 \text{ ms}$$

INLJ brilla cuando la outer table es chica — exactamente el caso típico en OLTP después de aplicar filtros.

INLJ: cuándo conviene

- Cuando S tiene un **índice en el atributo de join**
- Especialmente si el índice es **clustered** (las tuplas están contiguas)
- Si el índice es **unclustered** y hay muchos matches → costo puede explotar
- R debe ser la tabla **sin índice** (la outer); S la que tiene índice (la inner)

Resumen: tres algoritmos de join

Algoritmo	Costo	Requisitos
NLJ	$\text{Páginas}(R) + \text{Tuplas}(R) \cdot \text{Páginas}(S)$	Ninguno
BNLJ	$\text{Páginas}(R) + \lceil \text{Pág}(R) / (B - 2) \rceil \cdot \text{Páginas}(S)$	Buffer grande ayuda
INLJ	$\text{Páginas}(R) + \text{Tuplas}(R) \cdot \text{CostoBúsqueda}(S)$	Índice en S

Próxima clase (después de I1): Sort-Merge Join y Hash Join — algoritmos aún más eficientes que no requieren índice.

Ejercicio propuesto

Tenemos `Películas(pid, título, año, director)` con 5.000 páginas y `Actores_Películas(aid, pid)` con 2.000 páginas. Buffer de 50 páginas. Tuplas de 200 bytes en `Películas`, 50 bytes en `Actores_Películas`.

1. ¿Cuál es el costo de `Películas ⋈ Actores_Películas` con NLJ? ¿Cuál tabla conviene como outer?
2. ¿Y con BNLJ?
3. Si `Actores_Películas` tiene un B+ Tree de profundidad 2 en `pid`, ¿conviene INLJ?